16831 Robot Learning Midterm Report

Amir White
Mechanical Engineering
Carnegie Mellon University
United States
amirw@andrew.cmu.edu

Julian Millan
Mechanical Engineering
Carnegie Mellon University
United States
jmillan@andrew.cmu

Kayla McFarlane
Robotics Institute
Carnegie Mellon University
United States
kmcfarla@andrew.cmu.edu

1 Introduction

We propose to study deep reinforcement learning in a complex, real-time strategy game environment by training an autonomous agent to play Clash Royale. The project frames the game as a sequential decision-making problem in which the agent must learn to deploy cards, manage resources, and react to an opponent under partial observability and strict timing constraints. Using an Android emulator interfaced through Android Debug Bridge (ADB) and controlled via Python code, we construct a learning environment that exposes game state observations and accepts discrete, high-level actions. This setting provides a challenging testbed that goes beyond standard benchmarks, requiring the agent to reason over long horizons, stochastic opponent behavior, and delayed rewards, while remaining grounded in practical reinforcement learning methodologies. For the midterm report, we deploy an off-policy Q-function-based method utilizing Deep Q-Networks (DQN), and an on-policy policy-gradient method in the form of PPO.

2 Environment

The environment is within the Clash Royale mobile game. We will use the emulator "BlueStacks" to run the game on computers, allowing us to create ADB clients that have the agent interact with game. Clash Royale consists of a variety of screens on the home page, including a Battle screen, a deck editing and building screen, a Shop screen, and a Social screen. This agent will only be concerned with the Battle screen, and the match that occurs after the "Battle" button is pressed.

Each battle or match will be a one-on-one encounter, taking place on the "ladder" or the "Classic 1v1" game mode. The ladder is a ranked system where players collect "trophies" for winning battles, and lose trophies for losing battles. Collecting a certain amount of trophies allows the player to move up in "arenas" (the actual game areas where battles take place). Each arena has a unique appearance, and occasionally, a different layout (and therefore different state space). Moving up in arenas also unlocks new cards for the agent, adding further deck complexity as the agent's performance improves. Additionally, the game uses skill-based matchmaking, meaning as the agent increases trophies, so will the skill level of their opponent. Classic 1v1 mode was used for long stretches of training, as it is functionally the same as the ladder, but with no trophies or progression. Since Classic 1v1 is essentially a "free-play" mode, this allows the agent to play long stretches without the interruptions from the game.

2.1 State Space

The actual layout of each arena consists of a tiled gridspace. The arena is split into two sides along a horizontal median, demarcated by a "river", which acts as a null space and an area where cards cannot be played. During a match, players usually can only place most cards on their side of the arena. On each side of the arena, each player has three "crown towers". These are the fundamental win condition of the game. Each tower also works as a null space, as cards cannot be played directly on crown towers. The actual size of the arena is 32 tiles long at its longest point and 18 tiles wide [1].

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

However, to speed up the learning process when training the DQN model, this space was compressed into a coarse grid consisting of a 16x9 zone area (144 zones) representation, allowing faster training but less precise card placements. The arena contains two bridges that allow played cards to cross to the other side of the arena and attack the opponent's towers. The bridges are 2 tiles wide in Arenas 1-6 and 8-9; however, in Arenas 7 and 10-23, the bridges are 3 tiles wide. This slight difference should not make much of a fundamental difference in the performance of the agent.

Figure 1: Depiction of the screen while in a match.

This game information then gets represented for each agent via RGB data. We pull the pixel information from the screen via our ADB connection to the game emulator by taking screenshots in rapid succession. This information gets resized from the emulator resolution of 2560x1440 to be passed into the agent as an (84, 84, 3) vector per frame. In DQN, frame stacking of four frames is utilized, so the actual array size is (84, 84, 12). For DQN, this means that it has a final state space size of 84 height pixels * 84 width pixels * 12 stacked color channels, or 84,672 individual pixel values.

Using the screenshots from the emulator, other key information of the game can be determined by analyzing certain known pixels. These variables are also passed to the agent to provide more specific information and assist in automating the training process. For example, one can determine the end state of a match (whether a game was won or lost), the state of the current tower health, and other information like the elixir level of the agent.

2.2 Action Space

Essentially, the only way to interact with the state space is to play cards. Each player enters a match with a deck of eight cards, and a hand size of four cards. After a card is played, it goes to the back of the line, and the deck is cycled into the player's hand. This means that if a player plays Card A, they must play another four cards before Card A is back in their hand. We consider the initial action space partially observable, since the starting hand and deck order are randomized for each player. In the current game, there are 121 unique cards, and duplicates are not allowed in decks. This means the agent must be prepared for a variety of strategies and unique deck combinations.

There are three main types of cards: "troops", "spells", and "buildings". Troops are mobile units that will move towards any enemy or building (including towers) until defeated. Buildings are typically defensive units that are immobile and will expire after a certain amount of time. Spells are instantaneous sources of damage, slowing, or other types of effect. In order for an agent to take an action, it must select a card from its hand, and select a tile on the arena to play it. Buildings and troops are limited by the arena's divide, while spells can always be placed anywhere (including on towers) in the arena. We can thus consider the action space to be the transition of a card from its spot in the player's hand to a tile on the arena.

2.2.1 DQN

Recalling our compression of the arena size to 16x9 tiles, we then multiply by the four cards in our hand, add an extra action for "doing nothing" (not playing a card), to define a full action space consisting of 577 actions.

2.2.2 Policy Gradient

Unlike DQN, our PPO model uses the full gridspace, meaning it uses 32x18 tiles. Doing the same multiplication as above, we see that we end up with a full action space consisting of 2305 actions.

3 Reward Function

The primary considerations for the reward function design for each model were the difference in agent (blue) versus opponent (red) tower health and the win/loss status at the end of each match. The tower health of each side was represented to the agent as a percentage heuristic of the total health remaining for each side.

3.1 DQN

Once the agent receives the health percentage of both the blue and red sides, we then calculate the tower health from the previous environment step by storing and then updating a class-level variable in the environment. The calculated changes in tower health, denoted as "blue delta" and "red delta", are subtracted so that a net positive change in blue health yields a positive number and

net negative change results in a negative number. This number will always be between 0 and 1, and it is then multiplied by 100 and returned as the reward, meaning that nominally each step returns a reward between -100 and 100 per match. If an agent succeeds in maximizing this reward, it can generally learn to win in Clash Royale. However, since the game wins also depend on crown tower destruction, we also add a reward for the game end state, providing a return of 1000 for winning a match and -1000 for losing one. The environment is then reset so that this return cannot be overwritten by subsequent states and so that a new match can be started with an initial zero return. Ties are exceedingly rare and not considered.

3.2 Policy Gradient

The PPO reward function follows the same delta-based structure as DQN. Each step returns a reward proportional to the difference between the change in enemy tower health and the change in player tower health, scaled by 100. A terminal reward of +1000 is given for a win and -1000 for a loss. Reward calculation is skipped entirely for any step where elixir (to be described in § 4.2) reads as zero, since a zero elixir reading serves as a proxy for an end-game state transition, and the resulting health readings are noisy. Tower health values are also clamped to their previous values to prevent artificial healing artifacts caused by troops occupying a destroyed tower's pixel region.

4 Methods

4.1 Reward Function Modification

Our proposed reward function modification would tweak the rewards currently given, and add a tertiary reward marker to help steer the agent towards states that are winning, rather than just states with advantageous health differentials. Currently, if the agent plays a longer game, taking 30 actions, and remains in a strong average health differential of about 0.4, but loses at the end, it will still receive a net positive reward for that match and episode. Taking 30 actions at 0.4 health differential results in a return of 1200 for the episode based on tower health, and even losing this match still yields a positive return of 200. Increasing the win and loss returns by a factor of 2 to 3 would more strictly steer the agent to focus on winning more than anything else. Another way to shape the reward of the agent would be to include significant reward bonuses for destroying an enemy's crown tower, and major reward deductions for letting one's own crown tower get destroyed. The game is won and lost by whoever has the most crown towers remaining at the end of the match time, so this reward feature would emphasize the importance of crown towers to the agent and, therefore, emphasize the most important win condition.

4.2 State Space Modification

Another feature of the game is a resource called *elixir*. Elixir is a resource that is consumed in order to take an action during the game, and different actions cost different amounts. These actions almost always consist of playing cards. Elixir slowly builds over time, and any player can possess only a maximum of 10 elixir, so it is important to spend/save elixir wisely to win a game. If the agent attempts to take an action that requires more elixir than is currently available, then the action will fail and no progress will be made. Conversely, if the agent does not take actions despite having the maximum amount, it "leaks elixir" and effectively loses out on potential elixir it could have used later. Therefore, it would be wise to inform the agent of this resource so that it could learn to optimize its usage.

Currently, neither agent is aware of exactly how much elixir it has during training, other than knowing if it has zero elixir or non-zero elixir for ease of match queuing and action control (avoiding spamming actions during non-match screens, since a lack of elixir detected likely means we are not in a match). We can incorporate elixir into the state space to give the agents a better understanding of what cards they can play when, when their elixir is full and they should play a card, and how much elixir each card costs. This is an essential part of forming an optimal strategy for Clash Royale.

5 Results

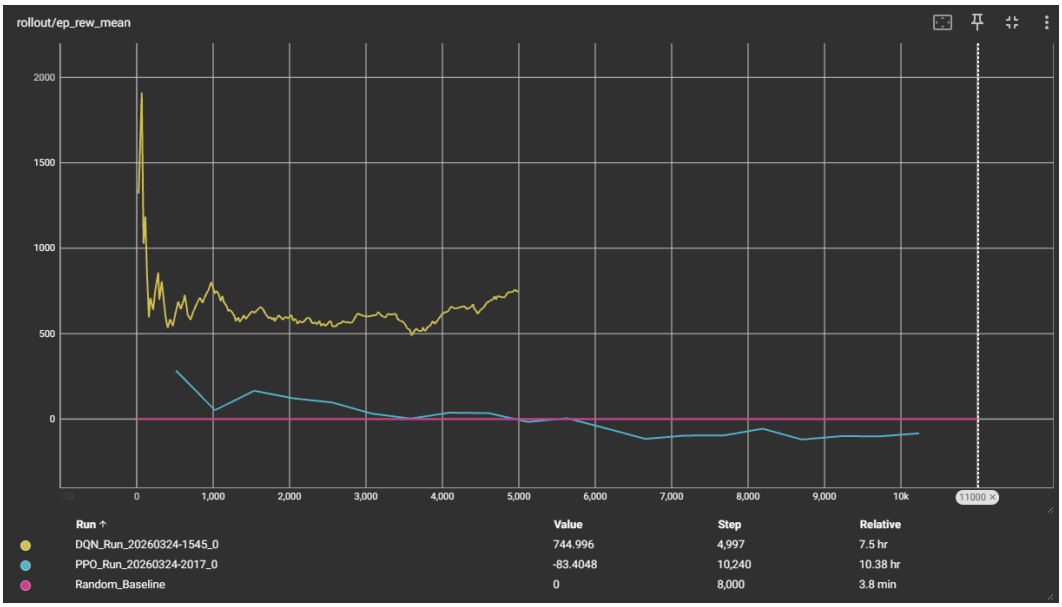

Figure 2: PPO, DQN, and Default Random Agent Comparison

5.1 DQN Analysis

We trained a DQN agent with a CnnPolicy on 84x84 RGB observations with four stacked frames and 577 discrete actions for 5000 steps. We initially defined the agent to use 100% exploration with a final exploration epsilon of 5% with an exploration fraction of 0.95. This means that 95% of the way through training, the agent would only be taking actions 5% of the time. The agent initially performed very well due to random actions at a lower trophy level (thus playing against worse human players). As exploration decreased, the agent settle around an average reward of about 500, good enough for semi-consistent winning and climbing up the ladder. Towards the end of training, as the agent climbed the ladder and relied more on its own learning, the agent started to steadily increase its average return. Given more training time, the agent could improve its average return even further and play Clash Royale even better.

The agent played the fundamental game fairly well. It started at the bottom of Arena 3 at 600 trophies, and by the end of training, reached a peak of 1150 trophies halfway through Arena 4. This is a marked advancement from the beginner level of the game to the early intermediate stage, when concrete strategies become more prevalent. During training, the agent was noticeably poor at using Spells, since these cards required precise timing and placement to be any effect. Additionally, Spells are uniquely able to instantly damage opponents' towers, providing a way for the agent to farm immediate reward despite not actually advancing towards winning the game. Spells became an action sink for the agent, and the card type actively hindered the agent's ability to learn the game. Another failure case was the situations where the agent could be ahead on tower health for the game, lose, and still net a positive reward for the episode (explained in § 4.1). This functionally worked as learning "losing" behavior.

5.2 Policy Gradient Analysis

We trained a PPO agent with a CnnPolicy on 84x84 RGB observations and 2305 discrete actions for 10,000 steps. Episode reward mean started near 175 and declined consistently to approximately -100 by the end of training, indicating the agent failed to learn a viable strategy. Since the agent

essentially had comparable performance to a random agent, the model didn't progress much in terms of trophies or arena progression, staying stagnant in the world of Clash Royale.

The primary cause of failure was a mismatch between the observation and action space. With 2305 possible actions and a pixel-based policy, the agent had insufficient information per step to explore the action space meaningfully. Most actions corresponded to card placements at positions the agent had no grounded spatial context for, causing the policy to degrade over time. Additionally, 10,000 steps represents a small fraction of the training data CNN-based policies typically require on game environments. Based on this, we simplified the setup to a 3-element state vector (elixir, player tower health, opponent tower health) and reduced the action space to 9 actions representing offensive and defensive card deployment at fixed positions, making the problem tractable within realistic training time.

5.3 Next Steps

Our next steps will include looking at how to implement the proposed modifications in (§ 4). Additionally, as the agents climb the trophy ladder, new cards and arenas will appear, allowing new decks and strategies for the agents to learn. We may find that the agents perform better with certain cards.

For the DQN method, we will work with the above as well as examine how different exploration rates can affect the model performance as it learns new cards, building a new model with a new state space. We will also explore tuning hyper parameters, such as learning rate, buffer size, and batch size.

For the policy gradient method, the next steps focus on addressing the fundamental mismatch between observation quality and action space size that caused the initial training run to degrade. The first planned change reduces the action space from 2305 discrete actions to 9 actions representing offensive and defensive deployment of each of the four cards in hand at fixed arena positions, plus a do-nothing action. This eliminates the spatial exploration burden from the policy and allows the agent to focus on learning when and which card to play based on the current game state. Paired with this, the observation is simplified from an 84x84 RGB screenshot to a 3-element state vector containing normalized elixir, player tower health, and opponent tower health. Together these changes make the problem tractable within practical training constraints, requiring on the order of 10,000 to 20,000 steps to observe meaningful learning as opposed to the millions of steps typically required for CNN-based pixel policies.

A secondary configuration replaces the 3-element state vector with a 42x42 single-channel grayscale screenshot, restoring some spatial context to the observation while keeping the same 9-action space. At this resolution the agent can observe rough troop positions and lane pressure, allowing it to make more informed placement decisions rather than committing to a fixed offensive or defensive position blind. This configuration is expected to require closer to 100,000 steps to converge and is planned as a follow-up run after the vector observation baseline is evaluated.

5.4 Website

The following link leads to out website showing videos of the performance of the various agents we trained playing Clash Royale: https://sites.google.com/andrew.cmu.edu/clashroyale16831/home

References

[1] Fandom Wiki Community. Arenas — Clash Royale Wiki. https://clashroyale.fandom.com/wiki/Arenas, 2025. Accessed: 2026-02-11.